Angular HandsOn - 5

Reactive Forms — FormBuilder, FormGroup, FormArray & Custom Validators

Aim:

To build a Reactive Form in Angular using FormBuilder, FormGroup, FormArray, built-in validators, custom synchronous validators, and asynchronous validators for the Student Course Portal.

Procedure:

1. Create the ReactiveEnrollmentForm component.
2. Import ReactiveFormsModule.
3. Create the form using FormBuilder.
4. Add built-in validators.
5. Implement a custom validator for the course code.
6. Implement an asynchronous validator for email availability.
7. Create a dynamic FormArray for additional courses.
8. Bind the form using formGroup and formControlName.
9. Display validation messages.
10. Submit the form and display the form values.

Codes:

reactive-enrollment-form.ts:

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import {
  ReactiveFormsModule,
  FormBuilder,
  FormGroup,
  Validators,
  FormArray,
  FormControl,
  AbstractControl,
  ValidationErrors,
```

```
  AsyncValidatorFn
} from '@angular/forms';
@Component({
  selector: 'app-reactive-enrollment-form',
  standalone: true,
  imports: [
    CommonModule,
    ReactiveFormsModule
  ],
  templateUrl: './reactive-enrollment-form.html',
  styleUrl: './reactive-enrollment-form.css'
})
export class ReactiveEnrollmentForm {
  enrollForm: FormGroup;
  constructor(private fb: FormBuilder) {
    this.enrollForm = this.fb.group({
      studentName: [
        '',
        [
          Validators.required,
          Validators.minLength(3)
        ]
      ],
      studentEmail: [
        '',
        [
          Validators.required,
          Validators.email
        ],
```

```
      [
        this.simulateEmailCheck()
      ]
    ],
    courseId: [
      '',
      [
        Validators.required,
        this.noCourseCode
      ]
    ],
    preferredSemester: [
      'Odd',
      Validators.required
    ],
    agreeToTerms: [
      false,
      Validators.requiredTrue
    ],
    additionalCourses: this.fb.array([])
  });
}
// Custom Validator
noCourseCode(control: AbstractControl): ValidationErrors | null {
  if (control.value && control.value.startsWith('XX')) {
    return {
      noCourseCode: true
    };
  }
```

```
  return null;
}
// Async Validator
simulateEmailCheck(): AsyncValidatorFn {
  return (control: AbstractControl) => {
    return new Promise<ValidationErrors | null>((resolve) => {
      setTimeout(() => {
        if (
          control.value &&
          control.value.includes('test@')
        ) {
          resolve({
            emailTaken: true
          });
        }
        else {
          resolve(null);
        }
      }, 800);
    });
  };
}
// Getter
get additionalCourses(): FormArray {
  return this.enrollForm.get('additionalCourses') as FormArray;
}
// Add Course
addCourse() {
  this.additionalCourses.push(
```

```
      new FormControl(
        '',
        Validators.required
      )
    );
  }
  // Remove Course
  removeCourse(index: number) {
    this.additionalCourses.removeAt(index);

  }
  // Submit
  onSubmit() {
    console.log('Form Value');
    console.log(this.enrollForm.value);
    console.log('Raw Value');
    console.log(this.enrollForm.getRawValue());
  }
}
```

reactive-enrollment-form.html:

```
<h2>Reactive Student Enrollment Form</h2>
<form
    [formGroup]="enrollForm"
    (ngSubmit)="onSubmit()">
    <!-- Student Name -->
    <div>
        <label>Student Name</label><br>
        <input
            type="text"
```

```
      formControlName="studentName">
    <br>
    <span
      class="error"
      *ngIf="
      enrollForm.get('studentName')?.touched &&
      enrollForm.get('studentName')?.errors?.['required']">
      Name is required
    </span>
    <span
      class="error"
      *ngIf="
      enrollForm.get('studentName')?.touched &&
      enrollForm.get('studentName')?.errors?.['minlength']">
      Minimum 3 characters required
    </span>
  </div>
  <br>
  <!-- Email -->
  <div>
    <label>Email</label><br>
    <input
      type="email"
      formControlName="studentEmail">
    <br>
    <span
      class="error"
      *ngIf="
      enrollForm.get('studentEmail')?.touched &&
```

```
      enrollForm.get('studentEmail')?.errors?.['required']">
      Email is required
    </span>
    <span
      class="error"
      *ngIf="
      enrollForm.get('studentEmail')?.errors?.['email']">
      Invalid Email
    </span>
    <span
      class="error"
      *ngIf="
      enrollForm.get('studentEmail')?.pending">
      Checking Email...
    </span>
    <span
      class="error"
      *ngIf="
      enrollForm.get('studentEmail')?.errors?.['emailTaken']">
      Email already exists
    </span>
  </div>
  <br>
  <!-- Course ID -->
  <div>
    <label>Course Code</label><br>
    <input
      type="text"
      formControlName="courseId">
```

```
    <br>
    <span
      class="error"
      *ngIf="
      enrollForm.get('courseId')?.errors?.['required']">
      Course Code is required
    </span>
    <span
      class="error"
      *ngIf="
      enrollForm.get('courseId')?.errors?.['noCourseCode']">
      Course code starting with XX is not allowed.
    </span>
  </div>
  <br>
  <!-- Semester -->
  <div>
    <label>Preferred Semester</label><br>
    <select formControlName="preferredSemester">
      <option value="Odd">
        Odd
      </option>
      <option value="Even">
        Even
      </option>
    </select>
  </div>
  <br>
  <!-- Agree -->
```

```
<div>
  <input
    type="checkbox"
    formControlName="agreeToTerms">
  I Agree to Terms & Conditions
  <br>
  <span
    class="error"
    *ngIf="
    enrollForm.get('agreeToTerms')?.touched &&
    enrollForm.get('agreeToTerms')?.errors?.['required']">
    You must accept the terms.
  </span>
</div>
<hr>
<!-- FormArray -->
<h3>Additional Courses</h3>
<button
  type="button"
  (click)="addCourse()">
  Add Another Course
</button>
<br><br>
<div formArrayName="additionalCourses">
  <div
    *ngFor="
    let ctrl of additionalCourses.controls;
    let i=index">
    <input
```

```
            [formControl]="ctrl"
            placeholder="Additional Course">
          <button
            type="button"
            (click)="removeCourse(i)">
            Remove
          </button>
          <br><br>
        </div>
    </div>
    <hr>
    <!-- Submit -->
    <button
      type="submit"
      [disabled]="enrollForm.invalid">
      Submit
    </button>
</form>
```

reactive-enrollment-form.css:

```
/* Page */
body {
    background-color: #f4f6f9;
    font-family: Arial, Helvetica, sans-serif;
}
/* Form */
form {
    width: 500px;
    margin: 30px auto;
```

```
    padding: 25px;
    border-radius: 10px;
    background-color: white;
    box-shadow: 0 3px 10px rgba(0,0,0,0.2);
}
/* Heading */
h2 {
    text-align: center;
    color: #1976d2;
}
/* Labels */
label{
    font-weight: bold;
}
/* Input */
input,
select{
    width:100%;
    padding:10px;
    margin-top:5px;
    margin-bottom:10px;
    border:1px solid #cccccc;
    border-radius:5px;
    box-sizing:border-box;
}
/* Validation */
.ng-valid.ng-touched{
    border:2px solid green;
}
```

```
.ng-invalid.ng-touched{
   border:2px solid red;
}
/* Buttons */
button{
   padding:10px 20px;
   margin-top:10px;
   margin-right:10px;
   border:none;
   border-radius:5px;
   cursor:pointer;
   background:#1976d2;
   color:white;
}
button:hover{
   background:#125ca1;
}
button:disabled{
   background:gray;
   cursor:not-allowed;
}
/* Error */
.error{
   color:red;
   font-size:13px;
}
/* Additional Courses */
h3{
   color:#444;
```

```
}
hr{
   margin:20px 0;
}
```

app.routes.ts:

```
import { Routes } from '@angular/router';
import { ReactiveEnrollmentForm } from './pages/reactive-enrollment-form/reactive-enrollment-form';
export const routes: Routes = [
 {
   path: '',
   redirectTo: 'enroll-reactive',
   pathMatch: 'full'
 },
 {
   path: 'enroll-reactive',
   component: ReactiveEnrollmentForm
 }
];
```

App.ts:

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
@Component({
 selector: 'app-root',
 standalone: true,
 imports: [RouterOutlet],
 templateUrl: './app.html',
 styleUrl: './app.css'
})
export class App {
```

```
}
```

App.html:

```
<h1 style="text-align:center;">
   Student Course Portal
</h1>
<hr>
<router-outlet></router-outlet>
```

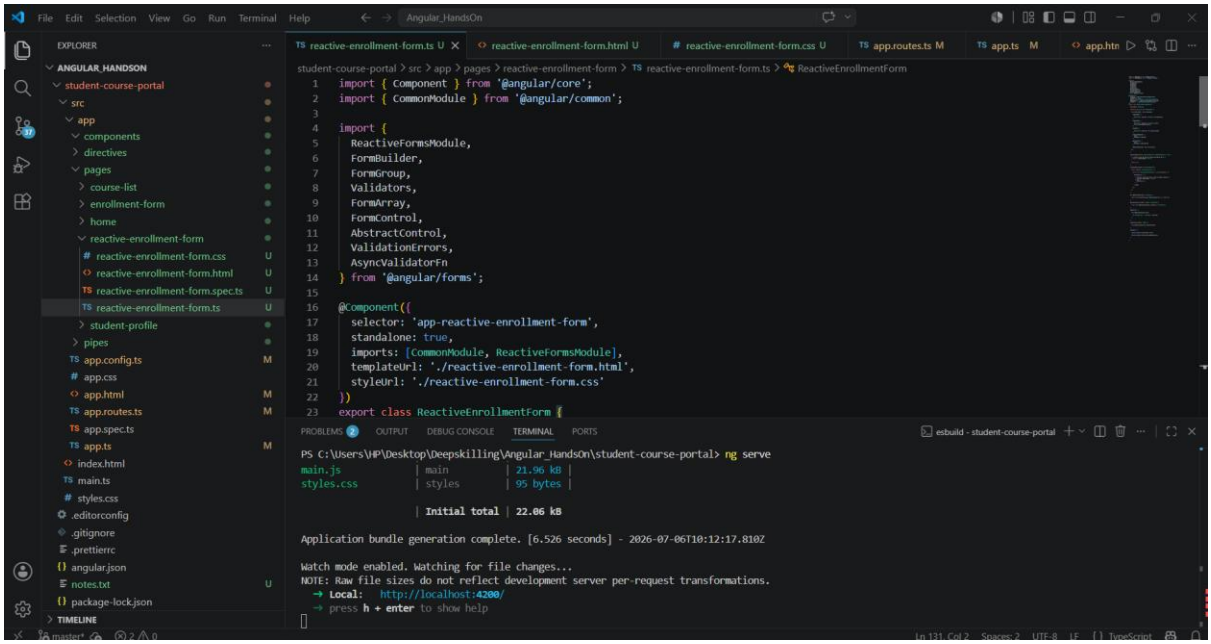
App.config.ts:

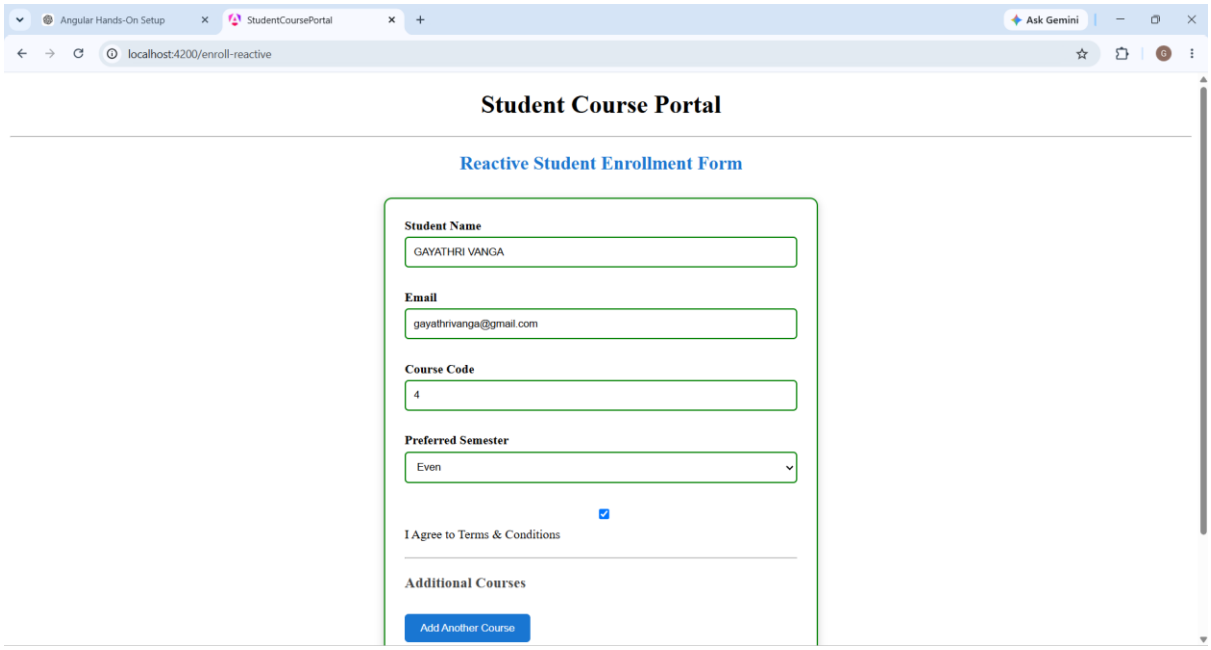
```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes)
  ]
};
```

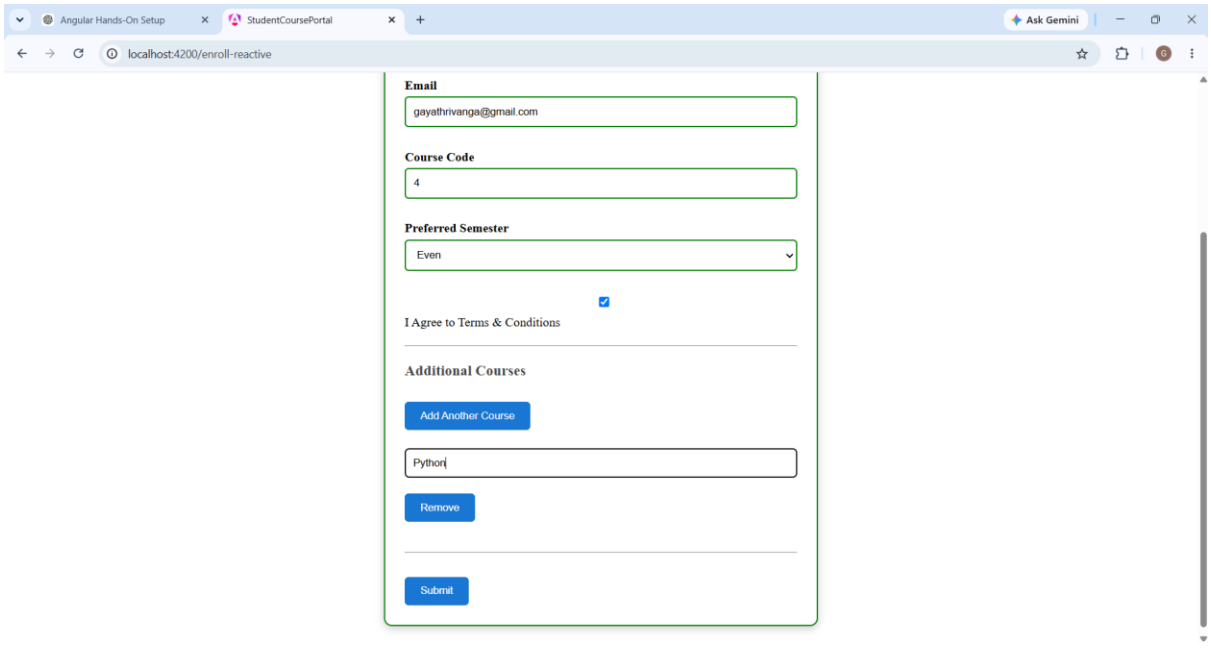
Result:

Successfully developed a Reactive Enrollment Form using Angular Reactive Forms with FormBuilder, FormGroup, FormArray, synchronous and asynchronous validation, and dynamic form controls.

Outputs:


File Edit Selection View Go Run Terminal Help
Angular_HandsOn
EXPLORER
ANGULAR_HANDSON
student-course-portal
src
app
components
directives
pages
course-list
enrollment-form
home
reactive-enrollment-form
reactive-enrollment-form.css
reactive-enrollment-form.html
reactive-enrollment-form.spec.ts
reactive-enrollment-form.ts
student-profile
pipes
app.config.ts
app.css
app.html
app.routes.ts
app.spec.ts
app.ts
index.html
main.ts
styles.css
.editorconfig
.gitignore
.prettierrc
angular.json
notes.txt
package-lock.json
TIMELINE
reactive-enrollment-form.ts U
reactive-enrollment-form.html U
reactive-enrollment-form.css U
app.routes.ts M
app.ts M
app.htn
student-course-portal > src > app > pages > reactive-enrollment-form > reactive-enrollment-form.ts > ReactiveEnrollmentForm
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import {
ReactiveFormsModule,
FormBuilder,
FormGroup,
Validators,
FormArray,
FormControl,
AbstractControl,
ValidationErrors,
AsyncValidatorFn
} from '@angular/forms';
@Component({
selector: 'app-reactive-enrollment-form',
standalone: true,
imports: [CommonModule, ReactiveFormsModule],
templateUrl: './reactive-enrollment-form.html',
styleUrl: './reactive-enrollment-form.css'
})
export class ReactiveEnrollmentForm {
PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
esbuild - student-course-portal
PS C:\Users\HP\Desktop\Deepskilling\Angular_HandsOn\student-course-portal> ng serve
main.js | main | 21.96 kB |
styles.css | styles | 95 bytes |
| Initial total | 22.06 kB
Application bundle generation complete. [6.526 seconds] - 2026-07-06T10:12:17.810Z
Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
➜ Local: http://localhost:4200/
➜ press h + enter to show help


Angular Hands-On Setup
StudentCoursePortal
Ask Gemini
localhost:4200/enroll-reactive
Student Course Portal
Reactive Student Enrollment Form
Student Name
GAYATHRI VANGA
Email
gayathrivanga@gmail.com
Course Code
4
Preferred Semester
Even
I Agree to Terms & Conditions
Additional Courses
Add Another Course


Angular Hands-On Setup
StudentCoursePortal
Ask Gemini
localhost:4200/enroll-reactive
Email
gayathrivanga@gmail.com
Course Code
4
Preferred Semester
Even
I Agree to Terms & Conditions
Additional Courses
Add Another Course
Python
Remove
Submit